Open Order Beta-Readiness Audit

Date: June 4, 2026
Scope: Read-only audit across cart, checkout, payments, fulfillment, SMS, RBAC, migrations, and tests.
Status: No code changes were made as part of this audit.

Architecture map

System layers

Layer	Primary modules	Identity
Cart	`cart-session-access`, `account-cart-ownership`, `cart-mutation-access-recovery`, `cart.service`, `cart.actions`	Session, `Cart.userId`, group actor
Checkout	`checkout/page.tsx`, `CheckoutForm`, `order.service`	Same + group fingerprint
Payments	`payment.service`, `post-payment.service`, Stripe webhook, `POST /api/orders`	PI metadata + idempotency
Vendor orders	`vendor-order-transition`, `order-status.service`, `routing.service`, `deliverect.service`	POS vs manual authority
Deliverect	`webhook-handler`, `deliverect-status-map`, reconciliation cron	HMAC + numeric status allowlist
SMS	`sms.service`, `sms-opt-out.service`, consent UI, Twilio inbound	`SmsOptOut` transactional consent
RBAC	`permissions.ts`, `admin-auth`, `vendor-dashboard-auth`	Membership + platform admin flag
Data	Prisma PostgreSQL, **53 migrations**, `Cart.userId` (Jun 2026)	—

Customer flow (summary)

1. **Cart:** Guest via `mennyu_session`; signed-in via `Cart.userId`; group via host/participant cookies.
2. **Checkout:** SSR validation → `POST /api/checkout` → `createOrderFromCart` → `createPaymentIntent`.
3. **Payment:** Stripe confirm → webhook `payment_intent.succeeded` → `processSuccessfulPayment`.

4. **Fulfillment:** `routing.service` → Deliverect or manual → VO status → parent order derivation.
5. **SMS:** Consent at checkout/account → milestone notifications via `sms.service`.

Test coverage by area

~275 `*.test.ts` files total. Strongest in `src/lib/` (~~139) and~~ ~~`src/services/`~~ ~~(~~54).

Area	Representative tests	Coverage quality
Cart ownership / mutations	`cart-session-access`, `account-cart-ownership`, `cart-mutation-access-recovery`, `cart.actions.auth`, `cart-api-auth`	**Strong** for access rules; weak E2E sign-in → checkout
Checkout validation	`order.service.cart-validation`, `cart-page-validation`, `checkout-api-auth`, group fingerprint tests	**Good** unit rules; no full `createOrderFromCart` integration
Stripe / payment	`payment-intent-order-validation`, `payment-intent-reuse`, `post-payment.service`	**Good** PI logic; **no** webhook route tests
Vendor order lifecycle	`order-state`, `order-status-deliverect-merge`, `vendor-order-next-action`	**Partial** — no `vendor-order-transition` or `routing.service` tests
Deliverect	`deliverect-status-map`, `webhook-handler`, `payload-validation`	**Good** mapping; no main webhook route or reconciliation job tests
SMS	`sms.service`, `customer-order-notification`, `sms-compliance`, Twilio inbound	**Good** compliance; thin DB-layer consent tests
RBAC / dashboards	`admin-layout-auth`, `admin-api-auth` (samples), `vendor-operational-api-auth`	**Weak** — no `permissions.ts` or layout auth tests
Migrations	None automated	**Gap** — manual `prisma migrate deploy`

Top 10 risk areas

#	Risk	Severity	Why it matters for beta

1	`createOrderFromCart` **solo auth bug**	**Critical**	Service re-check uses `groupOrderHostUserId` only (null for solo). Account carts with mismatched session can **fail checkout**.
2	**Stripe webhook untested in CI**	**High**	Production path unverified in automated tests.
3	**Pending order reuse skips re-validation**	**High**	`pending_payment` order returned without `validateCartForOrder`.
4	`vendor-order-transition` + `routing.service` **untested**	**High**	Core fulfillment branching has no dedicated tests.
5	**RBAC coarse + dev/legacy bridges**	**High**	`owner` vs `staff` not enforced; dev mode; `ADMIN_SECRET` and dashboard tokens.
6	**Dual cart mutation paths**	**Medium**	UI uses `cart.actions` (recovery); `/api/cart` has no recovery.
7	**Deliverect reconciliation job untested**	**Medium**	Cron fallback for stalled `sent` orders.
8	**TOCTOU between SSR checkout and API submit**	**Medium**	Page validates at load; API may reuse stale pending order.
9	`DEBUG_ADD_TO_CART_TRACE = true`	**Medium**	Verbose cart logging in `cart.actions.ts` / `cart.service.ts`.
10	**Migration deploy lag**	**Medium**	`Cart.userId` migrations must be applied before account-cart flows work.

Duplicate or inconsistent patterns

Pattern	Locations	Issue
Cart access	`assertCartSessionAccess` vs `checkout/page.tsx`	SSR duplicates solo checks
Cart mutations	`cart.actions` vs `/api/cart`	Recovery only in actions
Checkout auth	`api/checkout/route.ts` vs `createOrderFromCart`	Route passes `authUserId`; service uses `groupOrderHostUserId` only
Validation	`validateCartItemsForDisplay` vs `validateCartForOrder`	Collect-all vs fail-fast
Deliverect status mapping	`deliverect-status-map.ts` vs stub in `order-state.ts`	Dead duplicate

Dashboard auth	Layout vs API vs server actions	Similar rules in 3 layers; pod/vendor untested

---

Most critical files

File	Why
`src/lib/cart-session-access.ts`	Single cart gate
`src/lib/cart-mutation-access-recovery.ts`	Stale cart / account recovery
`src/services/order.service.ts`	`createOrderFromCart`, validation, pending reuse
`src/app/api/checkout/route.ts`	Checkout + PI creation
`src/services/payment.service.ts`	PI validation, record, reconcile
`src/services/post-payment.service.ts`	Paid → route → SMS → clear cart
`src/app/api/webhooks/stripe/route.ts`	Production payment finalization
`src/services/routing.service.ts`	Deliverect vs manual submit
`src/services/order-status.service.ts`	VO transitions + Deliverect inbound
`src/services/sms.service.ts`	Consent gate + Twilio send
`src/lib/permissions.ts`	Vendor/pod/admin gates
`prisma/schema.prisma + prisma/migrations/`	Schema truth for beta deploy

Recommended hardening order

1. Fix checkout auth for account-owned solo carts (`authUserId` in `createOrderFromCart`).
2. Re-validate or invalidate on pending checkout retry.
3. Stripe webhook + confirm integration tests.
4. Tests for `vendor-order-transition` and `routing.service` .
5. Deliverect webhook route + reconciliation job test.
6. RBAC regression suite (permissions, layout redirects).
7. Align or deprecate `/api/cart` mutations.
8. Gate `DEBUG_ADD_TO_CART_TRACE` behind non-production.
9. Pre-beta deploy checklist (`migrate status` , `SMS_MODE=twilio` , Deliverect sandbox).

10. Ops monitoring (webhook match failures, `routing_failure`, stuck `pending_payment`).

Suggested first patch

Pass `authUserId` through checkout order creation for solo account carts.

`createOrderFromCart` currently calls:

```
assertCartSessionAccess(cartId, mennyuSessionId, {
  authUserId: input.groupOrderHostUserId ?? null,
  mode: "checkout",
});
```

For solo carts, `groupOrderHostUserId` is undefined, so signed-in users with account carts fail when `mennyu_session` ≠ `cart.sessionId`.

Patch shape:

- Add `authUserId` to `CheckoutInput`.
- Pass `authSession?.user?.id` from `api/checkout/route.ts` for all checkouts.
- Use `authUserId: input.authUserId ?? input.groupOrderHostUserId ?? null` in `createOrderFromCart`.
- Add regression test for signed-in account cart with mismatched session.

Beta readiness snapshot

System	Beta-ready?	Notes
Cart ownership / mutations	**Mostly**	Checkout auth gap remains
Checkout validation	**Mostly**	Pending reuse + TOCTOU risks
Stripe finalization	**Conditional**	Webhook path unverified in CI
Vendor order lifecycle	**Conditional**	Transition/routing untested
Deliverect	**Conditional**	Webhook route + cron thinly tested

SMS consent / send	**Yes** (with ops)	Needs live Twilio A2P
RBAC	**Partial**	Coarse roles, legacy bridges
Migrations	**Deploy-dependent**	`Cart.userId` must be live
Test suite	**Good volume, uneven depth**	~275 tests

Overall: Architecture is coherent. Main beta blockers are checkout auth for account carts, payment webhook confidence, and fulfillment transition test gaps.